TP 2 - Python : Découverte des structures de données avec MicroPython

Ce TP a pour objectif de vous faire découvrir et manipuler les structures de données fournies par la bibliothèque standard du langage Python :

- Les listes
- Les tuples
- Les dictionnaires
- Les ensembles

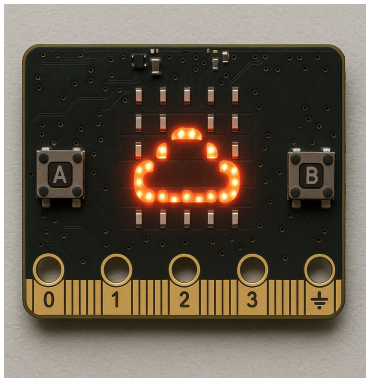

Figure 1: CloudFare NextGen

Outils à votre disposition

Liens et références utiles pour ce TP

- Documentation officiel de MicroBit via MicroPython : https://microbit-micropython.readthedocs.io/fr/latest/tutorials/hello.html
- La référence francophone de MicroPython : https://www.micropython.fr/reference/#/
- Site officiel de MicroBit : https://microbit.org/fr/
- Site officiel de MicroPython : https://micropython.org/
- Site français orienté débutant : https://python.doctor/
- Documentation officielle des structures de données Python : https://docs.python.org/fr/3.14/tutorial/datastructures.html

Éditeur de code

Pour ce TP nous utiliserons l'éditeur **Thonny** pré-installé sur vos machines. Pour fonctionner avec MicroPython il est **important de configurer l'interpréteur Python** via le menu :

`Tools > Options > Interpreter` et choisir **"MicroPython (BBC micro:bit)" à la place de "Local Python 3"**. **Enfin, dans l'onglet "General" : cochez "Use Tk file dialogs instead of Zenity".**

Pour fonctionner avec Python3 il est **important de configurer l'interpréteur Python** sur : **"Local Python 3"**

1 - Découverte des structures de données

Exercices pour découvrir les principales structures de données Python (à faire avec Python3 standard).

Exercice 1 - Listes

1. Recopier et exécuter le code suivant :

   ```
   notes = [12, 15, 9, 18]
   print("Notes :", notes)

   print(notes[0])
   print(notes[-1])

   notes[2] = 14
   notes.append(20)
   notes.remove(15)
   print("Final :", notes)
   ```

   Questions :

   - Différence entre `append()` et `insert()` ?
   - Effet de `notes.sort()` ?
   - Ajouter une note saisie au clavier.

2. Itération sur une liste :

   ```
   fruits = ["banane", "pomme", "poire"]
   for f in fruits:
       print(f)
   ```

   À faire : afficher uniquement les fruits contenant la lettre “a”.

3. Écrire une fonction retournant le minimum d’une liste sans utiliser `min()`.

Exercice 2 - Tuples

1. Tester :

   ```
   point = (10, 20)
   print(point)
   # point[0] = 30  # Que se passe-t-il ?
   ```

   Questions :

   - Peut-on modifier un tuple ?
   - Quand utiliser un tuple plutôt qu’une liste ?

À faire : créer une liste de tuples représentant des points et afficher ceux où `x == y` (pour cela utiliser la syntaxe de destructuration).

Exercice 3 - Dictionnaires

1. Tester :

```
etudiant = {"nom": "Doe", "prenom": "John", "age": 19}
print(etudiant)
print(etudiant["nom"])
etudiant["age"] = 20
etudiant["filiere"] = "CIEL"
print(etudiant)
```

   Questions :

   - Que renvoient `keys()`, `values()`, `items()` ?
   - Différence entre `etudiant["adresse"]` et `etudiant.get("adresse")` ?

2. Parcourir un dictionnaire :

```
for cle, valeur in etudiant.items():
    print(cle, valeur)
```

   À faire : afficher l'étudiant ayant la meilleure note dans `{"Alice":15, "Bob":12, "Eve":18}`.

Exercice 4 - Ensembles (sets)

1. Tester :

```
a = {1, 2, 3}
b = {3, 4, 5}

print(a | b)
print(a & b)
print(a - b)
```

   Questions :

   - Un set peut-il contenir des doublons ?
   - Le set conserve-t-il l'ordre des éléments ?

2. À partir de la liste `[1, 2, 2, 3, 4, 4, 5]`, produire un set contenant uniquement les valeurs uniques.

Exercice 5 – Lecture d'un fichier d'utilisateurs (username / password)

On souhaite traiter, avec Python, des données sous la forme : `username;password`

Cela permet de s'entraîner à lire un fichier ligne par ligne, à analyser les données et à manipuler les structures vues précédemment.

1. Création du fichier `users.txt` Créer un fichier texte contenant par exemple :

```
alice;azerty123
bob;passw0rd
charlie;123456
```

> Le fichier doit être dans le même dossier que votre script Python.

2. Lecture du fichier et construction d'un dictionnaire Écrire un programme qui lit ce fichier et construit un dictionnaire associant chaque `username` à son `password`.

```python
users = {}

with open("users.txt", "r", encoding="utf-8") as f:
    for ligne in f:
        ligne = ligne.strip()
        if ligne:
            username, password = ligne.split(";")
            users[username] = password

print(users)
```

Questions :

- Que contient le dictionnaire `users` après lecture ?
- Que se passe-t-il si deux lignes ont le même `username` ?
- Pourquoi un dictionnaire est-il adapté pour associer un identifiant à un mot de passe ?

3. Variantes : liste de tuples et set de tuples Adapter votre code pour obtenir :

a. Une liste de tuples :

```
users_list = []

with open("users.txt", "r", encoding="utf-8") as f:
    for ligne in f:
        ligne = ligne.strip()
        if ligne:
            username, password = ligne.split(";")
            users_list.append((username, password))
```

b. Un set de tuples :

```
users_set = set()

with open("users.txt", "r", encoding="utf-8") as f:
    for ligne in f:
        ligne = ligne.strip()
        if ligne:
            username, password = ligne.split(";")
            users_set.add((username, password))
```

Comparer les deux versions :

- Comment chaque structure réagit-elle aux doublons dans le fichier ?
- Quelle structure est la plus adaptée pour :
  - rechercher un mot de passe à partir d'un identifiant ?
  - détecter les doublons ?
  - supprimer un utilisateur au hasard ?

2 - Mini serveur DNS distribué avec MicroBit (CloudFare NextGen)

Principe

Le **D**omain **N**ame **S**ystem (Système de nom de domaine) ou DNS est un service informatique **distribué** qui associe les noms de domaine Internet avec leurs adresses IP.

Dans cet exercice, chaque micro:bit est un serveur DNS qui maintient une petite base locale d'associations (domaine → IP) et communique avec les autres via la radio.

Description du protocole :

- Format des messages : `OP;domaine;ip`
- OP peut prendre les valeurs suivantes : {ADD, `DEL`}
  - ADD est envoyé uniquement au démarrage, pour annoncer la table initiale (une opération ADD par enregistrement)
  - `DEL` est envoyé lorsqu'une suppression locale est effectuée (pour informer les autres)

Étapes :

- Au démarrage :
  1. Chaque carte lit son fichier `dns.txt`.
  2. Elle construit sa structure de données locale.
  3. Pour chaque entrée, elle envoie un message `ADD;domaine;ip` aux autres cartes.
- Pendant l'exécution :
  - Lors d'un appuie sur le bouton A :
    * choisir un enregistrement au hasard,
    * le supprimer localement,
    * afficher le domaine (via la matrice de LED),
    * envoyer `DEL;domaine;ip`.
  - À la réception d'un message :
    * ADD → ajouter l'entrée si elle n'existe pas encore,
    * `DEL` → supprimer l'entrée si elle existe,
    * afficher le domaine concerné.